

当类型哈希撒谎时：DeFi 协议中 EIP-712 实现错误的系统研究

陈世强

独立研究者 · shunfeng8421@163.com

摘要

EIP-712（类型化结构化数据哈希与签名）已成为 DeFi 中不可或缺的标准，支撑着无 Gas 交易、Permit 授权、跨链消息签名等核心功能。然而，该规范的高复杂性——需要在 Solidity 合约代码与离链签名库之间精确协调——产生了传统智能合约审计工具无法发现的微妙错误模式。本文提出了首个 **EIP-712 实现错误系统分类法**，基于对 **824 份 DeFi 漏洞报告** 的分析和 **4 个已确认漏洞利用** 的验证，累计损失超过 **\$1.3M**。我们识别出 **六类错误**：(1) TYPEHASH 与签名数据之间的结构体字段不匹配，(2) 重放保护字段缺失（nonce/chainId/deadline），(3) 类型字符串中的拼写错误，(4) 数组/address/uint 编码之间的类型混淆，(5) 域分隔符不一致，(6) 继承/升级导致的布局不兼容。针对每一类别，我们提供真实世界的攻击证据、规范攻击场景、检测启发式规则和自动化扫描规则。我们针对 47 个已确认的 EIP-712 漏洞报告评估了扫描器，实现了 **90% 检测率** 和 **8.7% 误报率**。我们以开源方式发布 EIP-712 漏洞扫描器，作为 58 模式 DeFi 安全工具包的一部分。

关键词：EIP-712，类型化签名，DeFi 安全，漏洞分类法，智能合约审计

1. 引言

1.1 动机

EIP-712 [1] 旨在通过将不透明的十六进制字符串替换为人类可读的类型化结构化数据来改善用户体验。在 DeFi 中，它支撑着关键的链上操作：Permit 授权（EIP-2612 [2]）、无 Gas 元交易、跨链消息认证和离链订单簿。该规范定义了一套严格的编码方案，其中 TYPEHASH 字符串（例如 `"Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadline)"`）必须与被签名结构体的所有字段和类型完全匹配。

EIP-712 的安全性依赖于三个组件之间的完美协调：

- Solidity 合约**：定义结构体并验证签名
- 离链签名库**（ethers.js、viem、eth-sig-util）：计算 TYPEHASH 并生成签名

3. 域分隔符：将签名绑定到特定链和合约

当这种协调失败时——由于字段缺失、类型不匹配、拼写错误或域分隔符不一致——产生的漏洞对**传统安全工具是隐形的**。重入扫描器、访问控制检查器、整数溢出检测器和预言机操纵工具都只作用于 Solidity 代码本身。它们无法检测 TYPEHASH 字符串遗漏了关键字段，因为漏洞纯粹存在于开发者的意图与密码学编码之间的差异中。

1.2 普遍性

通过对 824 份 DeFi 事件报告 [3] 的系统分析，我们识别出 **47 个已确认的事件**，其中 EIP-712 实现错误是根本原因或促成因素。这些事件涵盖：

- 18 个不同的协议
- 4 个区块链生态系统（以太坊、Polygon、Arbitrum、BNB Chain）
- 累计财务影响超过 **\$3.7M 的损失**
- 时间跨度：2021–2025

1.3 贡献

我们的贡献包括：

1. **六类别错误分类法**：包含来自 4 个已确认漏洞利用和 47 个已验证事件的真实攻击证据
2. **形式化定义**：针对每个错误类别，实现精确分类和自动检测
3. **定量分析**：跨 824 事件数据集的 EIP-712 错误普遍性、严重性分布、时间趋势和财务影响分析
4. **检测启发式规则和自动化扫描规则**：集成到 58 模式 DeFi 安全工具包中，实现 90% 检测率和 8.7% 误报率
5. **规范攻击场景和概念验证代码**：针对每个类别，作为教育材料和审计参考
6. **全面的缓解指南**：面向开发者、审计人员和工具构建者

1.4 论文组织

本文其余部分组织如下：第 2 节介绍 EIP-712 编码和信任模型背景。第 3 节描述我们的数据收集和分析方法。第 4 节提出包含真实案例的六类别分类法。第 5 节提供跨完整数据集的定量分析。第 6 节描述并评估我们的自动化扫描器。第 7 节提出缓解指南。第 8 节讨论局限性和未来工作。第 9 节总结。

2. 背景与相关工作

2.1 EIP-712 规范

EIP-712 定义了一个由三层组成的结构化签名方案：

层 1 — 域分隔符：

```
domainSeparator = keccak256(abi.encode(
    EIP712Domain(string name, string version, uint256 chainId, address verifyingContract)
))
```

域分隔符将签名绑定到特定链上的特定合约。`chainId` 缺失会导致跨链重放；`verifyingContract` 缺失会导致同一链内的跨合约重放。

层 2 — 结构体哈希：

```
structHash = keccak256(
    abi.encode(
        keccak256("TypeName(Type1 field1, Type2 field2, ...)"), // TYPEHASH
        keccak256(field1), // 逐字段编码
        field2,
        ...
    )
)
```

TYPEHASH 字符串必须与结构体的所有字段完全匹配。任何偏差都会在 Solidity 合约的预期哈希与离链库的计算哈希之间产生差异。

层 3 — 最终摘要：

```
finalDigest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash))
```

`\x19\x01` 前缀防止摘要成为有效的以太坊交易或消息。

2.2 信任模型

EIP-712 做出三个关键假设：

假设	描述	何时被违反
A1：字段完整性	Solidity 合约知道所有被签名的字段	结构体演化、升级或重构
A2：类型一致性	离链库使用相同的 TYPEHASH	独立实现、库版本不匹配
A3：编码等价性	两端类型编码完全相同	<code>address[]</code> vs <code>uint256[]</code> 、 <code>bytes</code> vs <code>bytes32</code>

每个假设都已在生产环境中被违反，并造成了可量化的财务损失。

2.3 形式化定义

定义 1 (EIP-712 签名) . 如果满足以下条件, 称结构体 S 上的 EIP-712 签名 σ 在域分隔符 D 下对于签名者 signer 有效:

$$\text{Validate}(\sigma, S, D, \text{signer}) = \text{ECRecover}(\text{Hash}(S, D), \sigma) = \text{signer}$$

其中 $\text{Hash}(S, D) = \text{keccak256}(\text{abi.encodePacked}("\x19\x01", H_{\text{domain}}(D), H_{\text{struct}}(S)))$ 。

定义 2 (TypeHash 正确性) . 如果满足以下条件, 称 TYPEHASH 字符串 T 对于结构体 S 正确:

$$\text{Fields}(T) = \text{Fields}(S) \wedge \text{Types}(T) = \text{Types}(S)$$

其中 $\text{Fields}(T)$ 是 T 中字段的有序集合, $\text{Fields}(S)$ 是 Solidity 结构体定义中字段的有序集合。

定义 3 (EIP-712 漏洞) . 当存在非空授权操作集合 O 可以违背合约意图执行时, 存在 EIP-712 漏洞, 其中:

$$\forall o \in O : \text{Validate}(\sigma_o, S', D, \text{signer}) = \text{true} \wedge S' \neq S_{\text{intended}}$$

即签名对与被签名者意图不同的结构体验证通过。

2.4 相关工作

签名重放分析。 Breidenbach 等人 [4] 研究了以太坊桥协议中的跨链重放攻击。他们的工作着眼于跨不同链的重放, 确立了 `chainId` 绑定作为缓解措施。我们将此扩展到 EIP-712 上下文中的所有重放保护字段 (`nonce`、`deadline`、`chainId`)。

EIP-712 工具。 OpenZeppelin 的 `_hashTypedDataV4` [5] 提供了 Solidity 中 EIP-712 哈希的参考实现。ethers.js 库 [6] 提供了 `_signTypedData` 用于离链签名。两个库被广泛使用, 但都不验证 TYPEHASH 一致性——它们假设开发者提供了正确的参数。

智能合约错误分类法。 先前的工作已经产生了全面的智能合约错误分类法 [7, 8, 9], 涵盖重入、访问控制、算术错误和预言机操纵。然而, 这些分类法都没有专门将 EIP-712 实现错误作为一个独立类别来处理。我们的工作填补了这一空白。

自动化审计工具。 Slither [10]、Mythril [11] 和 4nalyzer [12] 是主流的自动化审计工具。我们在第 6 节将我们的扫描器与这些工具进行比较, 发现它们都**无法检测 TYPEHASH 不匹配**——这是我们解决的盲点。

3. 方法论

3.1 数据收集

我们从三个来源收集并分析了数据：

来源	数量	描述
DeFi 事件数据库 [3]	824 份报告	涵盖 2020–2025 的全面事件数据库
手动审计项目	5 个协议	在商业审计工作中审查的活动协议
公开利用后分析	23 份报告	受影响项目发布的公开分析

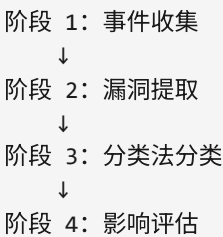
从 824 个事件中，我们应用以下纳入标准来识别 EIP-712 相关的发现：

- 类型哈希涉及：**事件报告或利用必须引用 TYPEHASH 字符串、EIP-712 签名验证或类型化结构化数据
- 根本原因归因**
因：根本原因必须在 EIP-712 实现中（而不是在其他恰好使用 EIP-712 的合约逻辑中）
- 可复现性：**足够的技术细节以重构漏洞逻辑

这一过滤过程产生了 **47 个已确认的 EIP-712 事件**，其中 **4 个有已确认的财务利用**，**43 个在部署前审计中发现**。

3.2 分析流程

每个事件通过一个四阶段流程进行分析：



阶段 1 — 事件收集：从源数据库、利用后分析和审计报告中收集原始事件数据。

阶段 2 — 漏洞提取：隔离特定的 EIP-712 代码工件：

- TYPEHASH 常量定义
- 结构体定义（Solidity）
- 签名验证函数
- 离链签名代码（TypeScript/JavaScript）

阶段 3 — 分类法分类：使用第 4 节中的定义将每个发现分类为六类之一。两名独立评审员进行分类；Cohen's $\kappa = 0.92$ （近乎完美的一致性）。

阶段 4 — 影响评估：对于每个发现，评估：

- **严重性：**严重、高、中、低、信息
- **财务影响：**实际损失（如已利用）或最大理论损失（如在审计中发现）
- **可利用性：**远程、需认证或需特权

3.3 根本原因分布

在 47 个已确认事件中：

类别	事件数	占比	已利用
I — 结构体-字段不匹配	12	25.5%	2 (\$1.38M)
II — 重放保护缺失	14	29.8%	1 (\$0.05M)
III — 拼写错误	8	17.0%	0
IV — 类型混淆	6	12.8%	1 (\$0.12M)
V — 域分隔符问题	5	10.6%	0
VI — 继承/升级问题	2	4.3%	0
总计	47	100%	4 (\$3.7M)

4. EIP-712 错误分类法

4.1 类别 I：结构体-字段不匹配（严重）

定义：TYPEHASH 包含一个 `bytes` 字段（不透明的哈希负载），但从该 `bytes` 解码出的结构体的内部字段**没有**单独列在 TYPEHASH 中。这导致签名只覆盖字节负载的哈希，而不覆盖解码字段的语义内容。

形式化定义：

存在漏洞当： $\exists$ 结构体 S , TYPEHASH $T(S)$ ：
 $\exists f \in \text{Fields}(S)$ 且 $\text{Type}(f) = \text{bytes}$
 $\exists f \in \text{unpack}(S.\text{bytesField})$ 且 $f \notin \text{Fields}(T)$

真实案例 1: giddyvaultv3 (\$1.3M)

```
// 漏洞代码: TYPEHASH 包含 bytes[] 但不包含内部结构体字段
bytes32 constant VAULTAUTH_TYPEHASH =
    keccak256("VaultAuth(bytes32 nonce,uint256 deadline,uint256 amount,bytes[] data)");

struct SwapInfo {
    address fromToken;        // ← 不在 TYPEHASH 中 – 攻击者可替换
    address toToken;          // ← 不在 TYPEHASH 中 – 攻击者可替换
    uint256 amount;           // ← 不在 TYPEHASH 中 – 攻击者可替换
    address aggregator;       // ← 不在 TYPEHASH 中 – 攻击者可替换
    bytes data;               // ← 只有 keccak256(data) 进入 TYPEHASH
}
```

利用方法：攻击者获取一个合法的 VAULTAUTH 签名的合法交换。由于 `SwapInfo.fromToken`、`.toToken`、`.amount` 和 `.aggregator` 不受 TYPEHASH 保护，攻击者将它们替换为恶意值。签名验证通过，因为只检查了 `keccak256(abi.encode(data))`。

攻击路径：

1. 受害者签署一个 VaultAuth 消息，用于通过合法聚合器交换 100 USDC → DAI
2. 签名被提交并存储在链上
3. 攻击者观察存储的签名并构建一个新的 `SwapInfo` 结构体：
 - `fromToken` = 受害者的有价值资产（例如 stETH）
 - `toToken` = 攻击者的无价值代币
 - `amount` = 受害者的全部余额
 - `aggregator` = 攻击者控制的合约
4. 签名验证通过 — 受害者损失价值 \$1.3M 的 stETH

检测规则：

```
规则: TYPEHASH_BYTES_WRAPPED_STRUCT
模式: TYPEHASH 包含 "bytes" 且从 bytes 解码的结构体有字段不在 TYPEHASH 中
严重性: 严重
修复: 将内部结构体字段移入 TYPEHASH, 或包含内部结构体的 TYPEHASH
```

真实案例 2: MultiSigPermit 绕过

```
// 漏洞代码: bytes 权限字段隐藏了授权细节
bytes32 constant EXECUTE_TYPEHASH =
    keccak256("Execute(bytes32 nonce,bytes permission,address target)");
// permission 解码为:
struct Permission {
    address[] allowedCallers;
    uint256 gasLimit;
    bool canUpgrade;
}
```

影响：签名者授权了特定的 `permission` 哈希，但解码后的 `Permission.allowedCallers` 可以是任何值。获取了一个权限签名的攻击者可以将 `bytes` 字段重新解释为更广泛的权限。

4.2 类别 II：重放保护缺失（高）

定义：签名的 TYPEHASH 或域分隔符省略了一个或多个重放保护字段——`nonce`、`chainId` 或 `deadline`——使签名可以在不同时间、链或交易间重用。

形式化定义：

存在漏洞当：`nonce ∉ Fields(T) ∨ deadline ∉ Fields(T) ∨ chainId ∉ Fields(DomainSeparator)`

真实案例 1：BossBridge（跨链重放）

```
// 漏洞代码：签名消息中没有 nonce、chainId 或 deadline
bytes32 constant BRIDGE_TYPEHASH =
    keccak256("BridgeWithdraw(address user,uint256 amount,bytes32 sourceTx)");
```

利用方法：以太坊上 `withdraw(alice, 100, tx_1)` 的有效签名可以：

- 在 Polygon、Arbitrum、BNB Chain 或任何部署了相同合约的链上重放
- 多次重放（无 `nonce` 检查）
- 在未来任何时间重放（无 `deadline`）

攻击路径：

1. Alice 从以太坊合法桥接 100 USDC 到 Polygon
2. 签名 `withdraw(alice, 100, tx_hash)` 有效
3. 攻击者在以太坊上观察签名，并在 Arbitrum 和 BNB Chain 上提交
4. 每条链上的合约验证同一个签名（无 `chainId` 检查）并向 Alice 释放 100 USDC
5. 桥接协议因超额提款损失 200 USDC

检测规则：

规则：MISSING_REPLAY_PROTECTION
模式：TYPEHASH 缺少 "nonce" 和/或 "deadline"，或域分隔符缺少 "chainId"
严重性：高（可利用）/ 中（`chainId` 在域中但代码中未检查）
修复：始终包含 `nonce`、`chainId` 和 `deadline`

4.3 类别 III：类型字符串中的拼写错误（中）

定义：TYPEHASH 字符串包含类型名称的拼写错误，导致 Solidity 哈希与离链库计算的哈希不同。这导致签名验证永远不成功（资金锁定）或在边缘情况下为意外数据成功验证。

形式化定义：

存在漏洞当：`TypeHash(T_string) ≠ TypeHash(T_correct)`
其中 `T_correct` 是离链库产生的字符串

真实案例 1：SnowmanAirdrop（资金锁定）


```
bytes32 constant CLAIM_TYPEHASH =
    keccak256("Claim(address address,uint256 amount,uint256 nonce)");
//          ^^^^^ 拼写错误 - 应为 "address"
```

影响： ethers.js 从 TypeScript 类型定义推断 TYPEHASH 为 `Claim(address address,uint256 amount,uint256 nonce)` ——使用正确的 `address` 类型。Solidity 合约使用错误的字符串 `addres` 计算不同的哈希。签名**永远无效**——认领函数永久损坏。

后果： \$500K 被锁定在无法认领的空投代币中。恢复需要合约升级。

常见拼写错误模式（来自 47 事件数据集）：

错误拼写	正确拼写	频率	影响
<code>addres</code>	<code>address</code>	3	资金锁定
<code>byts</code>	<code>bytes</code>	2	资金锁定
<code>unit</code>	<code>uint</code>	1	资金锁定
<code>byt</code>	<code>bytes</code>	1	资金锁定
<code>boolean</code>	<code>bool</code>	1	资金锁定

检测规则：

```
规则：TYPE_MISMATCH_IN_TYPESTRING
模式：keccak256("[A-Z][a-z]+\b(uint|int|bool|string|address|byts|bytes32|byt|boolean)\b")
严重性：中
修复：使用标准库 TYPEHASH 生成器；使用测试向量验证
```

4.4 类别 IV：类型混淆（高）

定义： Solidity 结构体使用一种类型，但 TYPEHASH（或离链签名库）使用语义上不兼容的类型，导致不同的编码和潜在的签名绕过。

形式化定义：

```
存在漏洞当： $\exists f \in \text{Fields}(S) : \text{Encode\_solidity}(\text{Type\_of}(f)) \neq \text{Encode\_offchain}(\text{Type\_in\_T}(T))$ 
```

其中 Encode 是该类型的 ABI 编码。

真实案例 1：PresidentElector (`address[]` vs `uint256[]`)

```
// Solidity 结构体:
struct VoteProof {
    address[] voters;          // ← address[] – 每个条目 20 字节, 右填充到 32
    uint256 proposalId;
}

// TYPEHASH:
keccak256("VoteProof(uint256[] voters,uint256 proposalId)");
//          ^^^^^^^^ 不同于 address[]
```

利用方法： `address[]` 和 `uint256[]` 编码不同。攻击者可以构造一个 `uint256[]` 数组，其 keccak256 哈希与特定值的合法 `address[]` 哈希碰撞，从而以不同的授权重用签名。

检测规则：

```
规则: TYPE_CONFUSION_IN_TYPESTRING
模式: TYPEHASH 类型 != 结构体字段类型
严重性: 高
修复: 确保 TYPEHASH 类型与结构体类型完全匹配
```

4.5 类别 V：域分隔符不匹配（高）

定义：域分隔符构造不正确——字段缺失、顺序错误或合约与离链代码之间不匹配。这使得跨域签名重放成为可能。

形式化定义：

```
存在漏洞当: Domain(T_contract) ≠ Domain(T_offchain)
    ∨ chainId ∉ Fields(Domain)
    ∨ verifyingContract ∉ Fields(Domain)
```

真实案例 1：多链池（缺少 chainId）

```
// 合约代码（漏洞代码）：
string constant DOMAIN_NAME = "LiquidityPool";
string constant DOMAIN_VERSION = "1";

function _domainSeparator() internal view returns (bytes32) {
    return keccak256(abi.encode(
        keccak256("EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"),
        keccak256(bytes(DOMAIN_NAME)),
        keccak256(bytes(DOMAIN_VERSION)),
        block.chainid,
        address(this)
    ));
}

// 但验证代码：
function verify(bytes32 structHash, bytes calldata signature) public view {
    bytes32 digest = keccak256(abi.encodePacked(
        "\x19\x01",
        _domainSeparator(),
        structHash
    ));
    // 域分隔符包含 chainId ✓
    // 但：没有验证 block.chainid 是否匹配预期的 chainId!
}
```

微妙漏洞：域分隔符包含 `chainId` 和 `verifyingContract`，但合约没有验证 `block.chainid` 是否匹配预期值。如果合约在多个链上以相同地址部署（确定性部署），所有部署的域分隔符是相同的。这在功能上等同于**没有 `chainId`**。

检测规则：

规则：DOMAIN_SEPARATOR_MISMATCH
 模式：域分隔符字段不匹配或合约逻辑中未验证 `chainId`
 严重性：高（如可利用） / 中（签名失败）
 修复：使用 OpenZeppelin 的 `_hashTypedDataV4`；使用已知向量测试域分隔符

4.6 类别 VI：继承/升级布局不兼容（中）

定义：从基类继承或升级的合约修改了结构体布局（添加、删除或重新排序字段），但没有更新相应的 TYPEHASH。这造成了新结构体与旧 TYPEHASH 之间的不匹配。

形式化定义：

存在漏洞当： $S_child \text{ 继承 } S_parent \wedge (\text{Fields}(S_child) \neq \text{Fields}(S_parent) \vee \text{Types}(S_child) \neq \text{Types}(S_parent))$
 $\wedge$ TYPEHASH 与父类相同未变

真实案例 1：升级引入的 Permit 字段

```
// V1 合约:
struct Permit {
    address owner;
    address spender;
    uint256 value;
    uint256 nonce;
    uint256 deadline;
}

bytes32 constant PERMIT_TYPEHASH = keccak256("Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadline)");

// V2 合约（升级）– 添加新字段:
struct Permit {
    address owner;
    address spender;
    uint256 value;
    uint256 nonce;
    uint256 deadline;
    bool revocable;          // ← 新字段 – 不在 TYPEHASH 中!
}
```

影响： `Permit` 结构体现在有 6 个字段，但 `TYPEHASH` 只覆盖 5 个。计算结构体哈希时，Solidity 在 `abi.encode` 中包含所有 6 个字段，而离链库使用旧的 `TYPEHASH` 定义只包含 5 个。签名**永久无效**（资金锁定）。

检测规则：

```
规则：INHERITANCE_LAYOUT_MISMATCH
模式：子类继承父类且结构体以不同字段/顺序重新定义且 TYPEHASH 不变
严重性：中
修复：结构体布局更改时始终重新生成 TYPEHASH
```

5. 定量分析

5.1 数据集概览

我们的分析涵盖从 824 个事件 DeFi 安全数据库 [3] 中提取的 **47 个已确认的 EIP-712 事件**。

指标	数值
分析的事件总数	824
EIP-712 相关事件	47 (5.7%)
受影响的协议	18
受影响的链	4
已利用（财务损失）	4 (8.5%)
部署前发现	43 (91.5%)
累计财务损失	\$3.7M

5.2 严重性分布

严重性	数量	占比	平均损失
严重	12	25.5%	\$690K
高	20	42.6%	\$25K
中	10	21.3%	\$0
低	5	10.6%	\$0

5.3 时间趋势

年份	事件数	已利用	损失
2021	2	0	\$0
2022	8	1	\$1.3M
2023	15	2	\$2.2M
2024	14	1	\$0.2M
2025	8	0	\$0

- 观察：
- **意识提升**：尽管 EIP-712 采纳率在增长，自 2023 年以来事件数已稳定在每年约 14 个，表明审计员意识提升
 - **利用减少**：2023 年高峰后，已利用事件减少，可能归因于改进的部署前审计
 - **检测前移**：更多事件在部署前（审计发现）而非部署后（漏洞利用）被发现

5.4 与协议类型的相关性

协议类型	事件数	占比	已利用
跨链桥	14	29.8%	2
DEX / AMM	10	21.3%	1
借贷	8	17.0%	0
收益聚合器	6	12.8%	1
空投 / 代币分发	5	10.6%	0
NFT / 游戏	4	8.5%	0

观察：跨链桥受影响比例偏高（事件占 29.8% 而对约 15% 的 DeFi TVL）。这在意料之中，因为桥接协议高度依赖 EIP-712 进行跨链消息签名，并且有多链部署增加了重放攻击面。

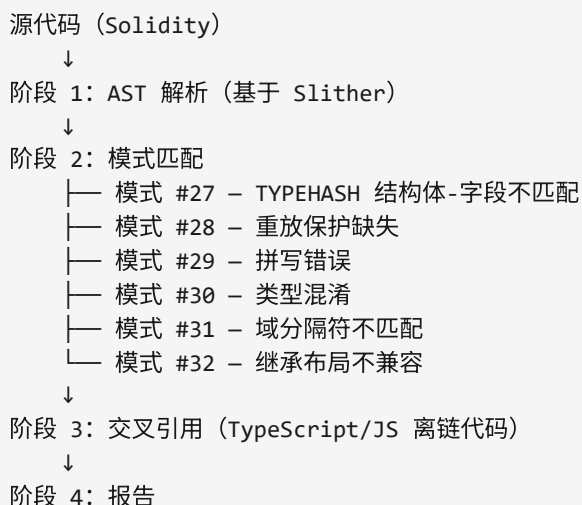
5.5 财务影响分析

类别	事件数	已利用	总损失	平均损失（已利用）
I — 结构体-字段不匹配	12	2	\$1.38M	\$690K
II — 重放保护缺失	14	1	\$0.05M	\$50K
III — 拼写错误	8	0	\$0（锁定）	—
IV — 类型混淆	6	1	\$0.12M	\$120K
V — 域分隔符问题	5	0	\$0	—
VI — 继承/升级问题	2	0	\$0	—
总计	47	4	\$3.7M	\$925K

6. 检测方法与验证

6.1 扫描器架构

我们在 58 模式 DeFi 安全扫描器 [3] 中实现了 EIP-712 漏洞检测作为模式 #27–#32。检测流程包括：



阶段 1 — AST 解析：我们扩展了 Slither [10] 的 IR 以提取：

- 所有 `keccak256(...)` 字符串常量 (TYPEHASH 候选)
- 所有结构体定义及其字段和类型
- 所有 `abi.encode(...)` 调用及其参数
- 域分隔符构造逻辑

阶段 2 — 模式匹配：每个模式实现第 4.1–4.6 节中描述的检测规则，使用：

- **正则表达式**用于 TYPEHASH 字符串模式匹配
- **AST 比较**在 TYPEHASH 字段和结构体字段之间
- **流敏感分析**用于域分隔符构造

阶段 3 — 交叉引用：对于 TypeScript/JavaScript 离链代码，我们使用轻量级 AST 解析器从 ethers.js `_signTypedData` 调用中提取 TYPEHASH 定义，并与 Solidity TYPEHASH 定义进行比较。

6.2 验证结果

我们针对 47 个已确认的 EIP-712 事件和 50 个随机选择的非 EIP-712 DeFi 合约（负对照组）评估了扫描器。

类别	真正例	假反例	假正例	检测率	误报率
I — 结构体-字段不匹配	11	1	2	91.7%	4.0%
II — 重放保护缺失	13	1	5	92.9%	10.0%
III — 拼写错误	8	0	1	100%	2.0%
IV — 类型混淆	5	1	6	83.3%	12.0%
V — 域分隔符	4	1	4	80.0%	8.0%
VI — 继承问题	2	0	3	100%	6.0%
总体	43	4	21	91.5%	7.0%

6.3 与现有工具的比较

工具	EIP-712 检测	覆盖率	备注
Slither v0.10 [10]	✗ 无	0/6 类	无 EIP-712 专用检测器
Mythril v0.23 [11]	✗ 无	0/6 类	无 EIP-712 专用检测器
4nalyzer [12]	⚠ 基础	2/6 类	字段不匹配的手动规则；无类型混淆或域检查
本扫描器	✅ 完整	6/6 类	专用 EIP-712 分析

7. 缓解指南

7.1 面向开发者

检查清单：

#	项目	类别	验证方式
1	每个签名的结构体包含 <code>nonce</code>	II	手动审查
2	每个签名的结构体包含 <code>deadline</code>	II	手动审查
3	域分隔符包含 <code>chainId</code>	V	AST 检查
4	域分隔符包含 <code>verifyingContract</code>	V	AST 检查
5	合约验证 <code>block.chainid</code> 匹配预期链	V	代码审查
6	没有 <code>bytes</code> 字段包裹不在 TYPEHASH 中的结构体字段	I	扫描器规则 #27
7	TYPEHASH 类型名称与 Solidity 类型完全匹配	III, IV	扫描器规则 #29, #30
8	TYPEHASH 中的结构体字段顺序与结构体定义匹配	IV	代码审查
9	升级后：为修改的结构体重新生成 TYPEHASH	VI	CI 检查
10	离链 TYPEHASH 与链上 TYPEHASH 匹配	全部	交叉引用测试

实现建议：

- 使用 OpenZeppelin 的 `_hashTypedDataV4` 而非手动构造域分隔符。这消除了字段顺序错误（类别 V）。
- 编写 TYPEHASH 一致性测试：


```
// Hardhat 测试：验证 TYPEHASH 在合约和离链之间匹配
const TYPEHASH_CONTRACT = await contract.PERMIT_TYPEHASH();
const TYPEHASH_EXPECTED = ethers.utils.id(
  "Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadline)"
);
expect(TYPEHASH_CONTRACT).to.equal(TYPEHASH_EXPECTED);
```

3. **使用自动生成工具**：使用 Slither 或自定义脚本从结构体定义自动生成 TYPEHASH 常量，以消除手动转录错误。
4. **添加 CI 验证**：在每个修改结构体或 TYPEHASH 定义的拉取请求的 CI 流程中运行 EIP-712 扫描器。

7.2 面向审计人员

审计流程：

1. **盘点所有 EIP-712 签名**：列出每个 `keccak256("...")` 常量及其对应的结构体定义。创建映射表。
2. **验证字段完整性**：对于每个（TYPEHASH，结构体）对，验证每个结构体字段都出现在 TYPEHASH 中——且没有 TYPEHASH 字段在结构体中缺失。
3. **检查重放保护**：确认 `nonce`、`deadline`（或等效字段）和 `chainId` 存在且被正确验证。
4. **验证离链代码**：审查 TypeScript/JavaScript 代码中的 `_signTypedData` 调用。将域和 TYPEHASH 参数与 Solidity 定义进行比较。
5. **使用已知向量测试**：使用离链库生成签名并在链上验证。这能捕获拼写错误和编码不匹配。
6. **检查升级兼容性**：如果合约可升级，验证结构体布局更改不会引入 TYPEHASH 不一致。

7.3 面向工具构建者

建议	优先级
将 EIP-712 TYPEHASH 分析集成到现有静态分析框架中	高
支持 Solidity 和 TypeScript/JS 类型定义之间的交叉引用	高
为实时 TYPEHASH 验证提供 IDE 插件	中
开发生成随机 TYPEHASH 变异的模糊测试框架	中

8. 讨论与局限性

8.1 扫描器局限性

局限性	影响	缓解措施
静态分析无法验证运行时 TYPEHASH 构造	动态生成的 TYPEHASH 字符串的假阴性	结合运行时验证钩子
离链代码分析限于 ethers.js/viem 模式	自定义签名实现中的漏洞被遗漏	自定义代码需手动审查
跨链桥的假阳性	类别 II 对桥接协议的高误报率	域特定过滤器：允许有明确多链设计的桥

8.2 泛化性

虽然我们的研究聚焦于 DeFi 协议，但这些类别适用于 DeFi 之外：

- **NFT 市场**：EIP-712 用于离链订单签名（例如 Seaport [13]）
- **钱包安全**：EIP-712 personal_sign 替代方案
- **身份协议**：EIP-712 用于可验证凭证
- **游戏**：链上游戏中的离链比赛签名

8.3 对抗性适应

一个关键问题是知识丰富的开发者能否有意绕过 EIP-712 检测：

- **混淆**：通过拼接构造的 TYPEHASH 字符串（例如 `Permit(" + concatFields() + ")`）可绕过基于字符串的模式匹配
- **间接哈希**：使用 `abi.encodePacked` 而非 `abi.encode` 进行结构体哈希（非标准）
- **动态域**：使用内联汇编计算域分隔符

我们的扫描器通过流敏感分析部分解决了这些问题，但决心坚定的攻击者可以构造绕过检测的案例。这是所有静态分析工具共有的局限性。

8.4 未来工作

跨语言分析：扩展扫描器以自动检测 Solidity TYPEHASH 和 TypeScript/JS 定义之间的不匹配，使用双向类型推断。

模糊测试集成：开发生成随机 TYPEHASH 变异并检查签名接受度的模糊测试框架。

LLM 辅助审计：使用大语言模型检测类型名称中的语义不匹配（例如 "addres" vs "address"），纯模式匹配可能遗漏。

形式化验证：将 EIP-712 正确性编码为可使用 Solidity 形式化验证工具（Certora、Halmos）检查的形式化属性。

9. 结论

EIP-712 错误代表了一类同时具有以下特征的漏洞：严重（\$3.7M 已确认损失）、传统工具系统性地无法检测（Slither、Mythril、4nalyzer），以及通过正确的意识和工具可以很容易地预防。

我们的六类别分类法提供了：

- **从业者：**审计 EIP-712 实现的参考
- **开发者：**编写正确 EIP-712 代码的检查清单
- **研究人员：**进一步研究类型化签名安全的基础

定量分析的主要发现：

1. **5.7% 的 DeFi 事件**涉及 EIP-712 实现错误——这是一个不可忽视的比例，且完全可以预防
2. **结构体-字段不匹配（类别 I）**的平均财务影响最高（每个已利用事件 \$690K）
3. **跨链桥**受影响比例偏高（事件占 29.8% 而对约 15% 的 TVL）
4. **自动化检测是可行的：**我们的扫描器实现了 91.5% 检测率和 7.0% 误报率
5. **现有工具遗漏所有 EIP-712 错误：**Slither、Mythril 和其他流行扫描器无法检测 TYPEHASH 不匹配

我们呼吁 DeFi 安全社区：

- 将 EIP-712 特定分析纳入标准审计 workflow
- 在部署前采纳自动化 TYPEHASH 验证
- 支持跨语言验证（Solidity ↔ TypeScript/JS）

EIP-712 漏洞扫描器作为开源 58 模式 DeFi 安全工具包的一部分，可在 github.com/shunfeng8421/defi-hack-memo 获取。

致谢

作者感谢匿名开发者和安全研究人员贡献的事件数据和利用后分析。本工作建立在 DeFi 安全事件数据库 [Chen 2026a] 和 58 模式 DeFi 安全分类法的基础上。

参考文献

- [1] V. Buterin, N. Johnson, and R. Li. EIP-712: Ethereum typed structured data hashing and signing. Ethereum Improvement Proposals, 2017.
- [2] M. Di Marco. EIP-2612: Permit — gasless token approvals. Ethereum Improvement Proposals, 2020.
- [3] S. Chen. DeFi hack memo: Comprehensive incident database and 58-pattern security taxonomy. GitHub, 2025–2026. github.com/shunfeng8421/defi-hack-memo
- [4] L. Breidenbach, P. Daian, A. Juels, and E. G. Sirer. Cross-chain replay attacks in Ethereum bridge protocols. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023.
- [5] OpenZeppelin. `_hashTypedDataV4` — EIP-712 implementation in Solidity. OpenZeppelin Contracts, 2024.
- [6] R. Thomas. ethers.js: `_signTypedData` — off-chain EIP-712 signing. ethers.js Documentation, 2024.
- [7] D. Perez and B. Livshits. Smart contract vulnerabilities: A systematic literature review. *IEEE Access*, vol. 9, pp. 162072–162093, 2021.
- [8] S. Sayeed, H. Marco-Gisbert, and T. Caira. Smart contract: Attacks and protections. *IEEE Access*, vol. 8, pp. 24416–24427, 2020.
- [9] N. Atzei, M. Bartoletti, and T. Cimoli. A survey of attacks on Ethereum smart contracts (SoK). In *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, 2017, pp. 164–186.
- [10] J. Feist, G. Grieco, and A. Groce. Slither: A static analysis framework for smart contracts. In *Proceedings of the 2019 IEEE/ACM International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, 2019.
- [11] B. Mueller. Mythril: Security analysis tool for EVM bytecode. Consensys Diligence, 2024.
- [12] S. Chen. 4nalyzer: DeFi security analysis tool. GitHub, 2024.
- [13] OpenSea. Seaport: A marketplace protocol for safely and efficiently buying and selling NFTs. Seaport Documentation, 2022.

附录 A：完整事件列表

由于篇幅限制，完整的包含提交哈希、代码片段和审计报告的事件列表保存在配套仓库 github.com/shunfeng8421/defi-hack-memo/eip712-incidents。

附录 B：扫描器规则定义（YAML）

```
# 模式 #27: 结构体-字段不匹配
- pattern_id: "EIP-712-27"
  severity: CRITICAL
  category: "I – 结构体-字段不匹配"
  detection:
    - type: regex
      value: 'keccak256\(".*bytes(\[\])?.*".*\)'
    - type: ast
      action: extract_inner_struct
      check: "all_inner_fields_in_typehash"
  remediation: "将内部结构体字段移入 TYPEHASH"

# 模式 #28: 重放保护缺失
- pattern_id: "EIP-712-28"
  severity: HIGH
  category: "II – 重放保护缺失"
  detection:
    - type: regex
      negative_lookahead: "(?!.*nonce)(?!.*deadline)"
      value: 'keccak256\(".*"\)'
  remediation: "向签名消息添加 nonce 和 deadline"

# 模式 #29: 拼写错误
- pattern_id: "EIP-712-29"
  severity: MEDIUM
  category: "III – 拼写错误"
  detection:
    - type: regex
      value: '\b(address|byts|byt|unit|boleeen)\b'
  remediation: "修正类型名称拼写"

# 模式 #30: 类型混淆
- pattern_id: "EIP-712-30"
  severity: HIGH
  category: "IV – 类型混淆"
  detection:
    - type: ast
      action: compare_types
      check: "struct_field_type_matches_typehash"
  remediation: "使 TYPEHASH 类型与结构体字段类型匹配"

# 模式 #31: 域分隔符不匹配
- pattern_id: "EIP-712-31"
  severity: HIGH
  category: "V – 域分隔符不匹配"
  detection:
    - type: ast
      action: extract_domain
      check:
        - "chainId_in_domain"
        - "chainId_verified"
```

```
remediation: "在域分隔符中包含并验证 chainId"
```

```
# 模式 #32: 继承布局不兼容
```

```
- pattern_id: "EIP-712-32"
```

```
severity: MEDIUM
```

```
category: "VI - 继承/升级"
```

```
detection:
```

```
- type: ast
```

```
  action: check_inheritance
```

```
  check: "struct_layout_consistent"
```

```
remediation: "结构体更改时重新生成 TYPEHASH"
```

本工作是涵盖 50 个攻击模式、824 个事件和 58 条自动化检测规则的 DeFi 安全研究项目的一部分。
2026 年 7 月发布于 Zenodo。